CS2613 Midterm Documentation Quick References

Python

1. Basic I/O

UserInput = input()

2. File I/O

Programming Question 9

f = open("file.txt", "r")

"r" - Read - Default value. Opens a file for reading, error if the file does not exist

"a" - Append - Opens a file for appending, creates the file if it does not exist

"w" - Write - Opens a file for writing, creates the file if it does not exist

"x" - Create - Creates the specified file, returns an error if the file exists

3. Strings

Removing all special characters from the string using "re.sub":

```
import re
#Remove all special characters
    cleaned = re.sub(r'[^a-zA-Z\n]', '', string)
```

String.split()

The split() method splits a string into a list.

You can specify the separator; default separator is any whitespace.

Note: When maxsplit is specified, the list will contain the specified number of elements plus one.

Syntax

string.split(separator, maxsplit)

Parameter Values

separator: Optional. Specifies the separator to use when splitting the string. By default, any whitespace is a separator maxsplit: Optional. Specifies how many splits to do. Default value is -1, which is "all occurrences"

4. **Lists**

   List Syntax

```
myList = [value1, value2, ...]
```

list() - create an mutable empty list if no arguement

5. **Arithmetic**
6. **Loops**
7. **Conditionals**
8. **Recursion**

JavaScript

1. **Basic I/O**

<u>Lab 2</u>

2. **File I/O**

   2.1 Reading a File Asynchronous

   <u>Check Lab 7</u>

   <u>Syntax:</u>

```
import { createRequire } from "module";
const require = createRequire(import.meta.url);
const fs = require('fs')
```

fs.readFile(path, options, callback);

<u>Example:</u>

```
fs.readFile('Lab7_StudentFiles/' + file, 'utf8', (err, data) => {
    if (err) {
```

```
        console.error(err);
    return;
    }
    //Do Whatever with the file data which is stored in 'data'

}
```

2.2 Reading Files Synchronously

Syntax:

fs.readFileSync(path, options);

In this syntax:

- **path:** A string representing the path to the file you want to read. This can be a relative or absolute path.
- **options (optional):** An object or string that specifies the encoding and flag. If not provided, it defaults to null for encoding (returning a buffer) and 'r' for the flag (open for reading).

Return Value:

- Returns the content of the file. If no encoding is specified, it returns a Buffer object; otherwise, it returns a string.

Example:

```
// Read the file synchronously
const data = fs.readFileSync('./input.txt', { encoding: 'utf8', flag: 'r' });

// Synchronously read 'input2.txt' with try-catch
try {
    const data2 = fs.readFileSync('./input2.txt', { encoding: 'utf8', flag:
'r' });
    console.log('input2.txt content:', data2);
} catch (err) {
    console.error('Error reading input2.txt:', err);
}
```

2.3 Reading JSON files
Check Lab 7

```
const parsed = JSON.parse(data)
```

3. **Strings**
4. **Lists**
5. **Arithmetic**
6. **Loops**
7. **Conditionals**
8. **Recursion**

Racket

1. **Basic I/O**
2. **File I/O**
3. **Strings functions**

   (substring str start [end])

   Returns a new **mutable** string that is (- end start) characters long, and that contains the same characters as str from start inclusive to end exclusive.

4. **Lists and Pairs**

(cons a b):
Create a new pair where first item is ‘a’, second item is ‘b’
If either ‘a’ or ‘b’ is a list, then output will be a list.
Eg: (cons “apple” “orange”) => ‘(“apple”. “orange”)
(cons “apple” (list “orange”)) => ‘(“apple” “orange”)

5. **Arithmetic**
6. **Loops**
7. **Conditionals**
8. **Recursion**
9. **Function declaration**
10. **Lambda**

Octave

1. **Basic I/O**
   input()

2. **File I/O**
   <u>Lab 9</u>
   fopen(“filename”, “mode”)

   fclose()

   fgetl() reads in a line

   fputs(fid, string)
   fid here is file input stream obtained from fopen()

3. **Matrices and Vertices Functions**
4. **Strings**
5. **Lists and Cell Array**

Cell array

cell(numRow, numCol)

Accessing Cell array

cell{row, col} for specific element or vector

cell(row, col) for subarray

6. **Arithmetic**
7. **Loops**
8. **Conditionals**
9. **Recursion**